APPy: Annotated Parallelism for Python on GPUs

Work in Progress

Motivation

Why GPU Acceleration Matters

Scientific Python workloads keep growing in scale and complexity:

- Simulation & numerical computing
- Data analysis & processing pipelines
- ML and non-ML scientific workloads

GPUs offer massive parallelism — but Python's default execution model is **sequential**.

The core problem

Most Python programs *could* benefit from GPU acceleration, but the path there is not easy.

Existing Path 1: Library-Based Acceleration

Examples: CuPy, PyTorch ops, JAX ops, cuBLAS

Pros

- Easy to use
- Pre-optimized kernels
- Great for standard linear algebra & tensor ops

Cons

- Many workloads cannot be expressed using predefined operators
- Irregular loops, custom control flow, stencil codes $\rightarrow$ not supported
- Must rewrite algorithms to fit the library's operator set

Existing Path 2: SIMT — Custom GPU Kernels

Examples: CUDA, Metal, OpenCL, Numba CUDA

Model: SIMT — each thread executes *independently* with its own control flow.

Pros

- Full control over execution
- Natural for irregular, control-heavy workloads (ray tracing, Mandelbrot)
- Potential for highest performance

Cons

- Must rewrite Python logic in a GPU kernel language
- Manual kernel launches & memory management
- Collective ops (reductions, dot products) require manual thread synchronization, barrier insertion, and shared memory management — error-prone and hard to maintain

Existing Path 3: SIMD-like — Triton

Triton's idea: write GPU kernels in a Python-like DSL that compiles to optimized GPU code.

Model: SIMD-like — a thread *block* collectively operates on a *tile* of data.

Pros

- High performance for dense, matrix/tensor-style workloads
- Minimal boilerplate vs. CUDA
- Strong auto-tuning infrastructure

Cons

- Requires learning Triton's DSL — not pure Python
- Still kernel-centric: users manage blocks, program IDs, tiled memory loads
- Awkward for SIMT-style code: irregular control flow, per-thread divergence (e.g. Mandelbrot) does not fit the tile/block model

The Gap in the Landscape

No simple way to express GPU parallelism using **regular Python loops** with **minimal annotation** that covers **both SIMT-style** (independent threads) **and SIMD-style** (collective thread-group ops) within the same Python front end.

	Triton	**CUDA/Metal**	**APPy**
SIMT-style code	×	✓	✓
Automatic collective ops	✓	×	✓
Auto host-device transfer	×	×	✓
No manual kernel launches	×	×	✓

APPy: Prior Work (CC 2024)

APPy v1 (CC 2024): Two-Layer Parallelism Model

Key idea: Annotate Python loops with pragmas; compiler generates GPU kernels.

```
#pragma parallel for        # outer loop -> grid (1 thread block per iteration)
for i in range(M):
    #pragma simd             # inner loop -> threads within that block
    for j in range(N):
        C[i, j] = A[i, j] + B[i, j]
```

Execution model:

- Outer `#pragma parallel for` → each iteration dispatches **one thread block**
- Inner `#pragma simd` → threads within the block work collectively
- Essentially a **group/SIMD-like model** — each outer iteration = a cooperative group

No CUDA, no Triton DSL required — works well for collective inner-loop workloads.

Where APPy v1 Hit a Limit

CC 2024's two-layer model (outer → thread block, inner → SIMD) is essentially a **group model only**. It cannot naturally express **flat SIMT** code.

Example: Mandelbrot — each pixel is fully independent, no inner loop to parallelize.

```
#pragma parallel for        # each iteration needs a WHOLE thread block?
for idx in range(width * height):
    # per-pixel work | no inner loop, no collective ops
    while abs(z) <= 2.0 and i < max_iter: ...
```

The bottleneck

Flat, shader-style SIMT code does not fit the two-layer model. Launching a full thread block per pixel is wasteful and unnatural. CC 2024 APPy lacks a first-class *individual* execution mode.

The Dual Programming Model

APPy v2: A Unified Dual Parallelism Model

One annotation. Two execution modes. Compiler decides.

Individual Mode *(SIMT-style)*
Each parallel iteration → one independent GPU thread.
Natural for irregular, control-heavy workloads.

"Assign each student a task. They work alone."

Group Mode *(SIMD-like)*
Each parallel iteration → one GPU threadgroup.
Threads inside the group coordinate on collective ops.

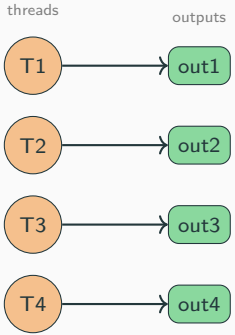
"Divide students into groups. Students in a group can collaborate."

Both modes use the *same* user-facing annotation:

```
#pragma parallel for
```

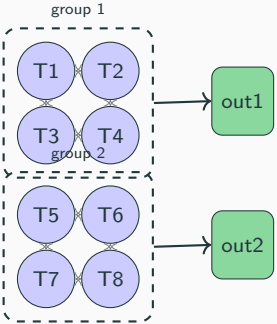
Individual vs. Group Mode: Visualized

Individual Mode



Each thread: independent task

Group Mode



Threads cooperate within a group

Individual Mode: Concept

Analogy

Assign each student an independent task. They work completely independently — no communication, no coordination. Each one just does their own thing.

On the GPU: each iteration of the `#pragma parallel for` loop maps to **exactly one GPU thread**.

Suitable for:

- Embarrassingly parallel workloads: Mandelbrot, ray marching, prime counting
- Workloads where iterations are fully independent
- At most a *final* global reduction across all iterations

Individual Mode: Mandelbrot Example

```
@appy.jit
def mandelbrot(out, width, height, max_iter):
    #pragma parallel for
    for idx in range(width * height):
        x = idx % width
        y = idx // width
        c = complex(x / width * 3.5 - 2.5,
                    y / height * 2.0 - 1.0)
        z, i = 0+0j, 0
        while abs(z) <= 2.0 and i < max_iter:
            z = z * z + c
            i += 1
        out[y, x] = i
```

Each pixel is computed by one independent thread. No communication between iterations.

Group Mode: Concept

Each `#pragma parallel for` iteration maps to **one GPU threadgroup**. Threads in a group share a whiteboard (shared memory) and coordinate via barriers.

The underlying mechanism is a **worksharing loop** (`#pragma workshare`): inner loop iterations are divided among threads; implicit barrier + reduction at the end.

Two user-facing styles — both desugar to the same worksharing loop:

- **Array expressions** — compiler rewrites to workshare loops automatically; concise for uniform, branchless ops
- **Explicit** `#pragma workshare` — write the loop directly; natural for irregular control flow

Group Mode: Symmetric Matrix Multiply (SYMM)

```
@appy.jit
def kernel_appy(alpha, beta, C, A, B):
    C *= beta
    for i in range(C.shape[0]):
        #pragma parallel for          <- launch one group per i
        for j in range(C.shape[1]):
            C[:i, j] += alpha * B[i, j] * A[i, :i]  # collective write
            temp2[j]  = B[:i, j] @ A[i, :i]          # dot product
        C[i, :] += alpha * B[i, :] * A[i, i] + alpha * temp2
    return C
```

Thread j handles column j — but within the group all threads cooperate on the slice operations and dot product via shared memory and threadgroup reductions.

Group Mode: Worksharing — Radial Binning

Radial binning — average data points falling in each radial bin.

```
#pragma parallel for          # one threadgroup per bin i
for i in range(npt):
    r1, r2 = rmax*i/npt, rmax*(i+1)/npt
    s, count = 0.0, 0
    #pragma workshare         # j-loop divided among threads
    for j in range(radius.shape[0]):
        if r1 <= radius[j] < r2:
            s += data[j]; count += 1
    # implicit barrier + reduction of s, count
    res[i] = s / count
```

- Each thread handles a stride of the `j`-loop
- The `if` branch is natural — no masking gymnastics needed
- `s`, `count` stay in registers; reduced at the implicit barrier

Memory Model

Variables created **inside** a `#pragma parallel for` loop body are **local to each loop iteration** — in both individual and group mode.

Storage is determined by how the variable is used:

- **Registers** — variable is always used as a whole (never indexed element-by-element)
- **Shared memory** — variable is ever indexed; implies one thread may read a value written by another thread, which requires shared memory

Individual mode exception

Only one thread owns each iteration — no inter-thread sharing. All variables stay in **registers**, even if indexed. Large arrays spill transparently via the backend compiler.

Group Mode: Memory Model — 1D Jacobi

1D Jacobi stencil — each output point averages its two neighbours.

```
#pragma parallel for
for i in range(0, n, K):          # one threadgroup per tile
    buf = np.zeros(K + 2)         # subscripted → shared memory
    acc = 0.0                     # never subscripted → register

    buf[1:K+1] = a[i:i+K]        # cooperative tile load
    buf[0]     = a[i - 1]        # left halo
    buf[K+1]   = a[i + K]        # right halo
    # implicit barrier | buf fully visible to all threads
    b[i:i+K] = (buf[:-2] + buf[1:-1] + buf[2:]) / 3.0
```

`buf` is in shared memory because thread `k` reads `buf[k-1]` and `buf[k+1]` written by neighbouring threads. `acc` is never shared — each thread keeps its own register copy.

Group Mode: Worksharing

The underlying mechanism in group mode is a **worksharing loop**: the inner loop is divided among threads, with an implicit barrier and reduction at the end.

Array expressions are **syntactic sugar** — the compiler desugars them into explicit `#pragma workshare` loops before codegen.

Users can also write `#pragma workshare` directly when the operation involves **irregular control flow** that is unnatural to express as array ops:

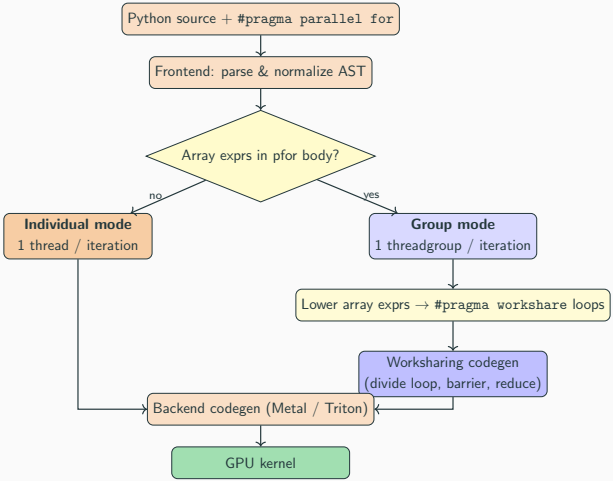
- Conditional accumulation with complex branching
- Multi-pass algorithms where later passes depend on earlier results
- Variable-length inner ranges (e.g. sparse row iteration)

Individual vs. Group Mode: Side by Side

	Individual	**Group**
pfor iter maps to	1 thread	1 threadgroup
Thread coordination	None	Shared memory + barriers
Inner mechanism	—	`#pragma workshare` loop
Inner notation	—	array exprs (sugar) or explicit workshare
Control flow	Per-thread	Arbitrary
Barrier at end	No	Implicit
Suitable for	Independent tasks	Collective ops per iteration
Examples	Mandelbrot	TRMM, SYMM, radial binning
Annotation	`#pragma parallel for`	

Compiler Design

Compiler Pipeline



Supported Workloads & Examples

NPBench Kernels Supported

Kernel	Mode	Key operation	Annotation
SAXPY / vec-add	Individual	Pointwise	`#pragma parallel for`
GELU	Individual	Pointwise	`#pragma parallel for`
Mandelbrot	Individual	Irregular	`#pragma parallel for`
GESUMMV	Group	`np.sum`	`#pragma parallel for`
GEMVER	Group	`@` (dot)	`#pragma parallel for`
TRMM	Group	`@` (dot)	`#pragma parallel for`
SYMM	Group	`@` + scatter	`#pragma parallel for`
Gramschmidt	Group	`@` (dot)	`#pragma parallel for`
SYRK / SYR2K	Group	Slice accumulate	`#pragma parallel for`
Jacobi-1D/2D	Group	Stencil	`#pragma parallel for`

Writing APPy vs. NumPy Reference

NumPy reference (TRMM):

```
for i in range(B.shape[0]):
    for j in range(B.shape[1]):
        B[i, j] += np.dot(A[i+1:, i], B[i+1:, j])
B *= alpha
```

APPy kernel:

```
@appy.jit
def kernel(alpha, A, B):
    for i in range(B.shape[0]):
        #pragma parallel for
        for j in range(B.shape[1]):
            B[i, j] += A[i+1:, i] @ B[i+1:, j]
    B *= alpha
```

The APPy version is essentially the NumPy version with one added pragma.

Progressive Optimization Path

Individual → Group: A Natural Upgrade Path

A key benefit of APPy's unified model: users can **start simple** and **upgrade incrementally** — using the same annotation.

Example: Stencil computation

Step 1 — Individual mode (simple, correct):

```
#pragma parallel for
for idx in range(N * N):   # each thread computes one output element
    i, j = idx // N, idx % N
    out[i, j] = 0.25 * (A[i-1,j] + A[i+1,j] + A[i,j-1] + A[i,j+1])
```

Step 2 — Group mode (tiled, better data locality):

```
#pragma parallel for        # each group handles one tile row
for i in range(0, N, TILE):
    out[i:i+TILE, :] = 0.25 * (A[i-1:i+TILE-1, :] + A[i+1:i+TILE+1, :]
                               + A[i:i+TILE, :-1]   + A[i:i+TILE, 1:])
```

Preliminary Results

Benchmark Setup

- **Platform:** Apple M3 (MacBook Air) — Metal GPU backend
- **Benchmarks:** NPBench kernels (PolyBench + custom) — all float32
- **Baselines:** NumPy, Numba (best mode per kernel)
- **Metric:** Median wall-clock time (ms), 5 runs
- APPy uses **a single** `#pragma parallel for` per kernel

Results: Individual Mode — Irregular Workloads (paper size)

Kernel	NumPy	Numba	APPy Metal	vs. NumPy
`mandelbrot` (1000×1000, iter=200)	1407 ms	239 ms	**3 ms**	**469**×
`prime_counting` (N=10^6)	2274 ms	—	**21 ms**	**108**×

Why such large speedups?

Each iteration is *fully independent* with *irregular control flow* (early exit, variable work per thread). NumPy/Numba cannot vectorise these efficiently. APPy launches one GPU thread per pixel/number — pure individual mode.

Results: Group Mode — Dense Linear Algebra (paper size)

Kernel	NumPy	Numba	APPy Metal	vs. NumPy
`syr2k`	3968 ms	975 ms	64 ms	**62**×
`syrk`	1458 ms	540 ms	31 ms	**47**×
`gemver`	237 ms	141 ms	31 ms	**7.6**×
`gesummv`	249 ms	117 ms	158 ms	1.6×
`symm`	4448 ms	673 ms	2064 ms	2.2×
`trmm`	1605 ms	292 ms	1309 ms	1.2×
`gramschmidt`	75 ms	26 ms	155 ms	0.5×

Takeaway

Memory-bandwidth-bound kernels (`syrk`, `syr2k`, `gemver`) see large GPU speedups. Loop-heavy kernels with sequential data dependencies are harder to parallelise.

Results: Stencil Kernels — Dispatch Overhead (M size)

Kernel	NumPy	Numba	APPy Metal	vs. NumPy
`fdtd_2d` *(M)*	34 ms	61 ms	113 ms	0.30×
`jacobi_2d` *(M)*	45 ms	30 ms	116 ms	0.39×
`floyd_warshall` *(M)*	41 ms	38 ms	253 ms	0.16×
`floyd_warshall` *(paper, N=2800)*	17527 ms	3140 ms	4461 ms	**3.9**×
`jacobi_1d` *(M)*	60 ms	110 ms	10704 ms	0.006×

Bottleneck: per-kernel dispatch overhead (≈240 μs/dispatch)

Stencil time-step loops require one GPU dispatch per step (no global sync in Metal). `jacobi_1d`: 4000 steps × 2 dispatches = 8000 launches. Measured: ≈165 μs Metal overhead + ≈85 μs actual GPU compute per step.

Discussion & Future Work

What We've Shown So Far

- A unified programming model that supports two GPU parallelism paradigms with a **single annotation**
- A compiler that automatically infers the mode and generates correct, efficient GPU code
- Support for compound reduction expressions (`+= np.sum(...)`, `@` inside expressions)
- Backends: **Metal** (macOS) and **Triton** (NVIDIA/Linux)
- Validated on a range of NPBench kernels

Limitations & Open Questions

- **1D array expressions only** inside pfor bodies — no 2D slice ops yet
- **No auto-tuning** for threadgroup sizes (currently fixed)
- `#pragma simd` is an internal annotation that could be renamed `#pragma group` to better reflect the semantics
- How to handle **memory allocation** for intermediate arrays inside pfor bodies?
- Formal correctness argument for the guard pass is informal today

Future Work

- **Rename** `#pragma simd` → `#pragma group`: SIMD is too narrow — group captures that threads *cooperate*, not just execute in lockstep
- **2D and higher-dimensional array expressions** inside pfor loops
- **Common backend passes**: consolidate shared logic (e.g. `ExtractReductionSubexprs`) rather than duplicating per backend
- **Performance evaluation** on NPBench against NumPy, CuPy, and Triton baselines
- **Auto-tuning** for threadgroup size and inner loop tile size
- **Worksharing sub-mode** (`#pragma workshare`): implement compiler support for dividing an inner loop among threads in a group, with implicit barrier and reduction at loop end. Enables algorithms with irregular per-element control flow that are awkward to express as array expressions — e.g. **bitonic sort** (cooperative in-place sort of a shared-memory tile across multiple compare-and-swap passes) and **sparse matrix-vector multiply** (variable-length row iteration in CSR format)

Summary

APPy's key contribution

A Python GPU programming model with a **single pragma** that supports two complementary parallelism paradigms — individual and group mode — without requiring users to think about threads, blocks, or shared memory.

Individual mode: "Each student works alone on their task."

Group mode: "Students form groups and collaborate."

Both expressed with `#pragma parallel for`.

Questions & Feedback Welcome

Backup Slides

Group Mode: TRMM (Triangular Matrix Multiply)

```
@appy.jit
def kernel_appy(alpha, A, B):
    M, N = B.shape
    for i in range(M):
        #pragma parallel for
        for j in range(N):
            B[i, j] += A[i+1:, i] @ B[i+1:, j]  # dot product per (i,j)
    B *= alpha
    return B
```

The `@` (dot product) inside the pfor body is a collective operation — the group of threads cooperates to reduce over the slice dimension.

Individual Mode: Prime Counting Example

```
@appy.jit
def count_primes(count, N):
    #pragma parallel for
    for n in range(2, N):
        is_prime = True
        for d in range(2, n):
            if n % d == 0:
                is_prime = False
                break
        if is_prime:
            count[0] += 1   # final reduction: one atomic add per thread
```

Each thread independently checks one number — irregular control flow handled naturally. `count[0] += 1` is an atomic global reduction, efficient here because **primes are rare** so few threads contend on the atomic add.

Individual Mode: When to Upgrade to Group Mode

Limitation of per-thread atomic reductions

If the reduction is *unconditional* (every thread contributes), N concurrent atomic adds cause high memory contention — inefficient.

Solution: upgrade to group mode.

- Each threadgroup first reduces locally (fast, no global traffic)
- Then one atomic add per group to the global counter
- $\lceil N/\texttt{GROUP_SIZE} \rceil$ atomic ops instead of N

Same `#pragma parallel for` annotation — just rewrite the body to use an array expression, and the compiler handles the rest.